21.Write a program to check whether a given number is a prime number.
Verify the output using white-box testing with JUnit assertions.

CODE:

```
package npkb;

import org.junit.Test;

import static org.junit.Assert.*;

public class npkb {

    public boolean isPrime(int n) {

        if (n <= 1) {

            return false;

        }

        for (int i = 2; i <= Math.sqrt(n); i++) {

            if (n % i == 0) {

                return false;

            }

        }

        return true;

    }

        @Test

    public void testPrimeTrue() {

        assertTrue(isPrime(13));

    }

    @Test

    public void testPrimeFalse() {

        assertFalse(isPrime(12));

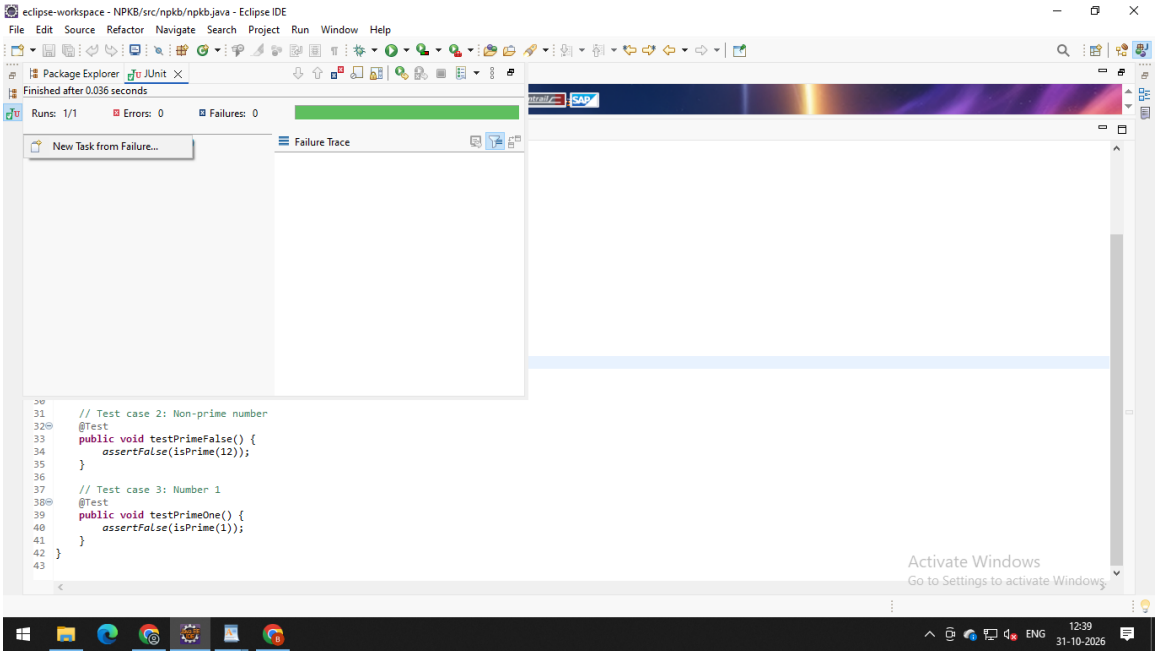
    }

    @Test

    public void testPrimeOne() {
```

```
        assertFalse(isPrime(1));
    }
}
```

OUTPUT:



S. No.	Test Case	Input(n)	Expected Result
1	TC1 – Prime Number	13	True
2	TC2 – Non-Prime Number	12	False
3	TC3 – Number One	1	False

22.Write a JUnit test case to reverse a number and validate the output using white-box testing techniques.

CODE:

```
package npkb;

import org.junit.Test;

import static org.junit.Assert.*;
```

```java
public class npkb {

    public int reverseNumber(int n) {

        int rev = 0;

        while (n != 0) {

            int digit = n % 10;

            rev = rev * 10 + digit;

            n = n / 10;

        }

        return rev;

    }

    @Test

    public void testReverseNormal() {

        assertEquals(321, reverseNumber(123));

    }

    @Test

    public void testReverseWithZero() {

        assertEquals(21, reverseNumber(120));

    }

    @Test

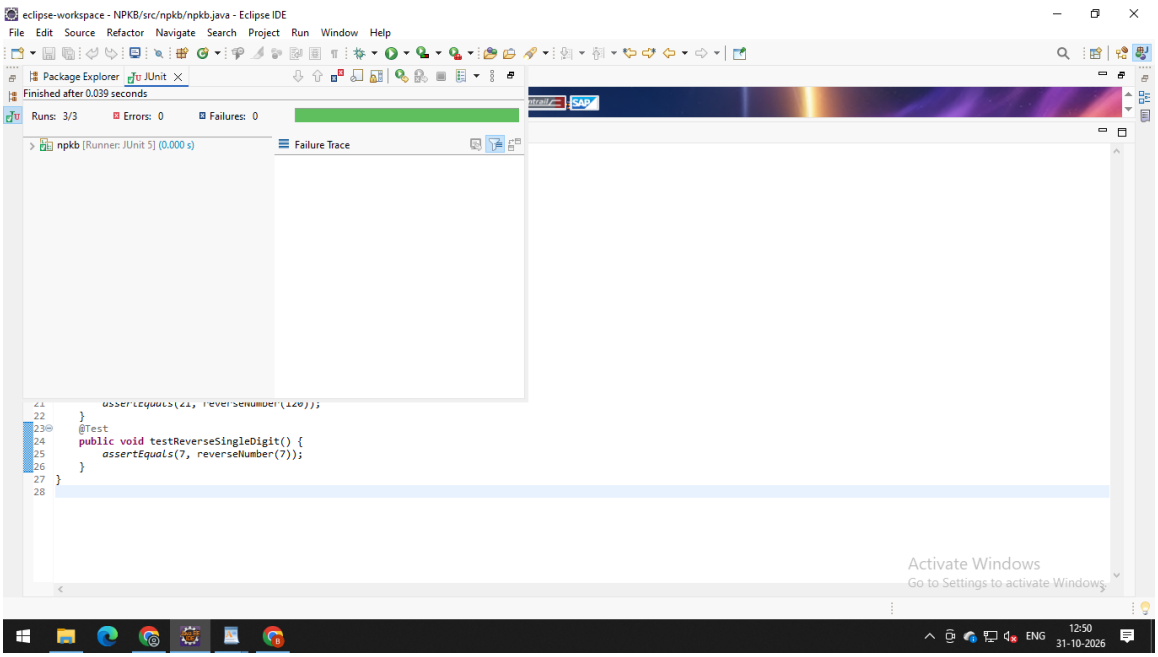
    public void testReverseSingleDigit() {

        assertEquals(7, reverseNumber(7));

    }

}
```

OUTPUT:



REPORT TABLE:

S. No.	Test Case	Input(n)	Expected Result
1	TC1 – Normal Number	123	321
2	TC2 – Number Ending with Zero	120	21
3	TC3 – Single Digit Number	7	7

23.Write a program to find the sum of digits of a given number and verify the output using white-box testing.

CODE:

```
package npkb;

import org.junit.Test;

import static org.junit.Assert.*;

public class npkb {

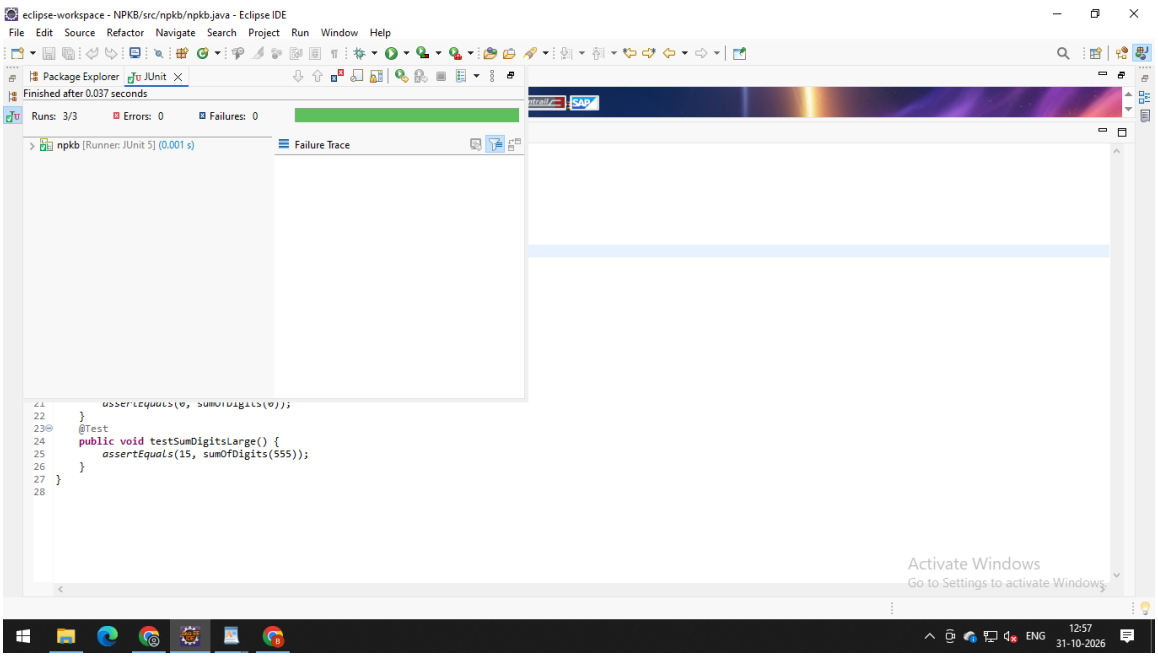
    public int sumOfDigits(int n) {

        int sum = 0;

        while (n != 0) {
```

```
            int digit = n % 10;
            sum = sum + digit;
            n = n / 10;
        }
        return sum;
    }
    @Test
    public void testSumDigitsNormal() {
        assertEquals(6, sumOfDigits(123));
    }
    @Test
    public void testSumDigitsZero() {
        assertEquals(0, sumOfDigits(0));
    }
    @Test
    public void testSumDigitsLarge() {
        assertEquals(15, sumOfDigits(555));
    }
}
```

OUTPUT:



REPORT TABLE:

S. No.	Test Case	Input(n)	Expected Result
1	TC1 – Normal Number	123	6
2	TC2 – Zero	0	0
3	TC3 – Large Number	555	15

24.Write a JUnit program to check whether two strings are equal ignoring case sensitivity, and validate the output using assert statements.

CODE:

```
package npkb;

import org.junit.Test;

import static org.junit.Assert.*;

public class npkb {

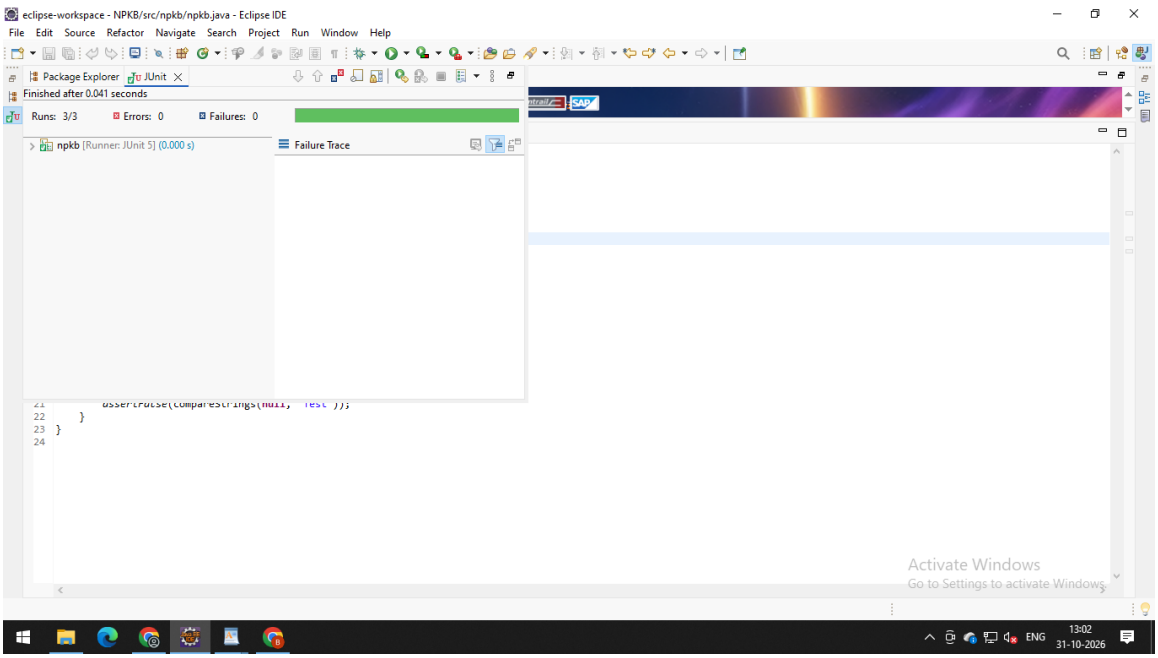
    public boolean compareStrings(String s1, String s2) {

        if (s1 == null || s2 == null)

            return false;
```

```
        return s1.equalsIgnoreCase(s2);
    }
    @Test
    public void testStringEqual() {
        assertTrue(compareStrings("Java", "java"));
    }
    @Test
    public void testStringNotEqual() {
        assertFalse(compareStrings("Java", "Python"));
    }
    @Test
    public void testNullString() {
        assertFalse(compareStrings(null, "Test"));
    }
}
```

OUTPUT:



REPORT TABLE:

S. No.	Test Case	Input(n)	Expected Result
1	TC1 – Equal Strings	Java, java	True
2	TC2 – Unequal Strings	Java, Python	False
3	TC3 – Null String	null, test	False

25.Write a program using functions to calculate the grade of a student based on marks.
Verify the output values using white-box testing.

CODE:

```
package npkb;

import org.junit.Test;

import static org.junit.Assert.*;

public class npkb {

    public String calculateGrade(int marks) {

        if (marks >= 90)

            return "A";

        else if (marks >= 75)

            return "B";

        else if (marks >= 50)

            return "C";

        else

            return "F";

    }

    @Test

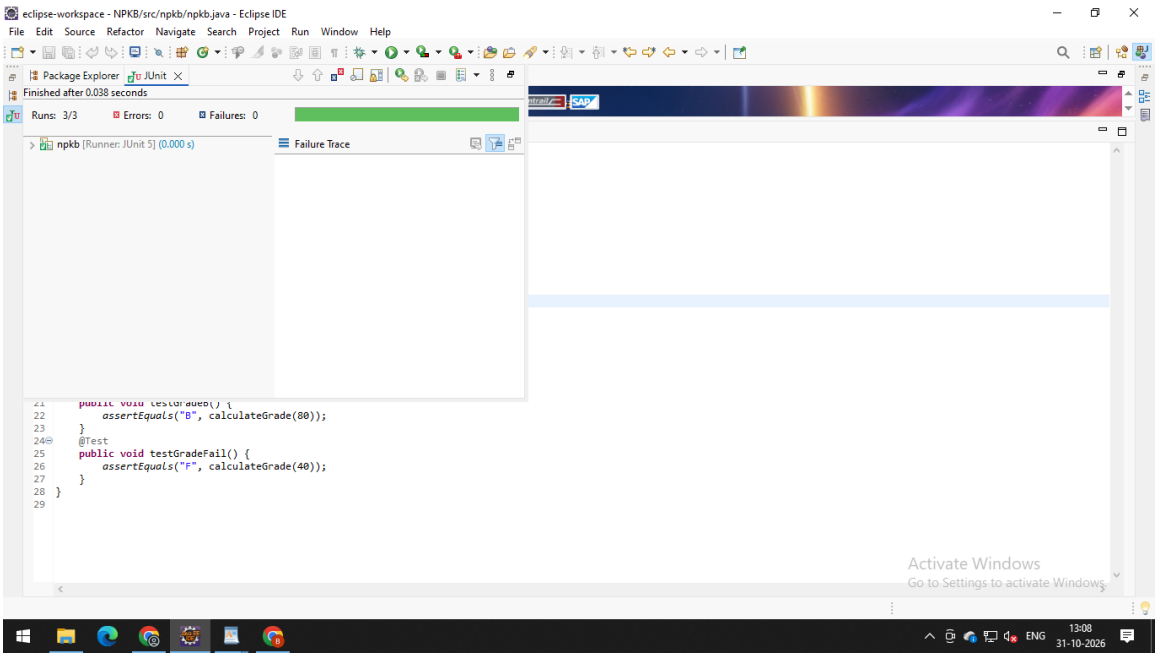
    public void testGradeA() {

        assertEquals("A", calculateGrade(95));

    }
```

```
    @Test
    public void testGradeB() {
        assertEquals("B", calculateGrade(80));
    }
    @Test
    public void testGradeFail() {
        assertEquals("F", calculateGrade(40));
    }
}
```

OUTPUT:



REPORT TABLE:

S. No.	Test Case	Input(n)	Expected Result
1	TC1 – Grade A	95	A
2	TC2 – Grade B	80	B
3	TC3 – Fail Grade	40	F